

# Jalon 4 — Dossier de Conception de l'Application & Architecture

---

**Projet** : OptiFleet — Plateforme de gestion de flotte automobile **Auteur** : Kahil Mokhtari **Formation** : CDA — Concepteur Développeur d'Applications **Échéance** : 30/04/2026 **Dépôt Git** : <https://github.com/Hakkai19z/optifleet-app->

---

## Table des matières

---

1. [Introduction](#)
  2. [Diagrammes de cas d'utilisation \(Use Case UML\)](#)
  3. [Diagrammes de séquence](#)
  4. [Diagramme de classes](#)
  5. [Architecture multi-couches \(Chapitre VIII\)](#)
  6. [Composants externes et bibliothèques](#)
  7. [Schémas complémentaires](#)
- 

## 1. Introduction

---

Ce dossier présente la **conception technique** d'OptiFleet : les diagrammes UML qui modélisent les interactions, la logique d'exécution et la structure interne de l'application, ainsi que la description de l'architecture multi-couches.

OptiFleet est une application de **gestion de flotte automobile** permettant à une organisation de :

- gérer un parc de véhicules (caractéristiques, statut, localisation GPS, quota kilométrique) ;
- **affecter les véhicules aux conducteurs** (cœur métier) ;
- suivre les entretiens et générer des alertes automatiques ;
- offrir à chaque conducteur une vue dédiée à son véhicule.

L'application repose sur une architecture **API REST Symfony 7 + SPA React 18**, conformément au socle technique imposé.

---

## 2. Diagrammes de cas d'utilisation (Use Case UML)

---

### 2.1 Acteurs du système

Le système distingue trois acteurs, organisés en **hiérarchie de rôles** (un rôle supérieur hérite des droits des rôles inférieurs) :

| Acteur         | Description                         | Hérite de    |
|----------------|-------------------------------------|--------------|
| Conducteur     | Utilise un véhicule affecté         | —            |
| Gestionnaire   | Gère la flotte et les affectations  | Conducteur   |
| Administrateur | Gère les utilisateurs et le système | Gestionnaire |

2.2 Diagramme de cas d'utilisation global



## Système OptiFleet

S'authentifier (JWT)

Consulter mon véhicule

Consulter mon quota km

Voir la localisation GPS

Signaler un problème

Supprimer mon compte (RGPD)

Gérer les véhicules (CRUD)

Affecter un véhicule

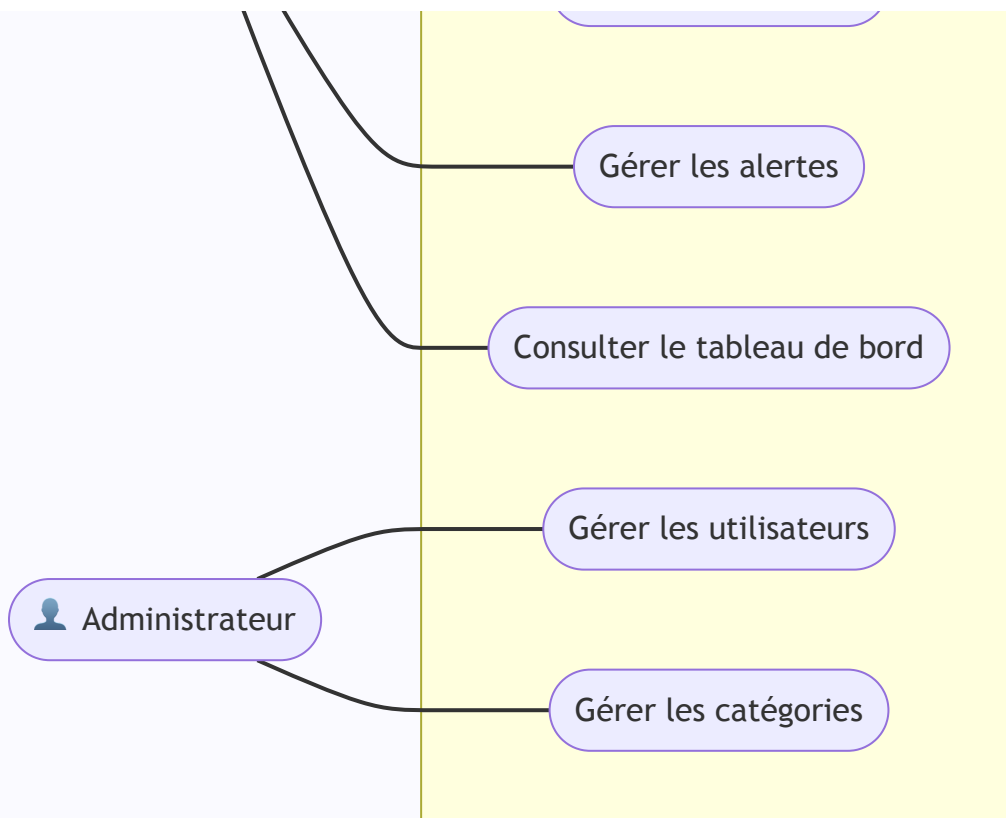
Libérer un véhicule

Consulter la vue flotte

Gérer les entretiens

 Conducteur

 Gestionnaire



**Note sur l'héritage des acteurs :** le Gestionnaire bénéficie de tous les cas d'utilisation du Conducteur, et l'Administrateur de tous ceux du Gestionnaire (relation de généralisation entre acteurs). Pour la lisibilité, seuls les cas **spécifiques** à chaque rôle sont reliés sur le schéma.

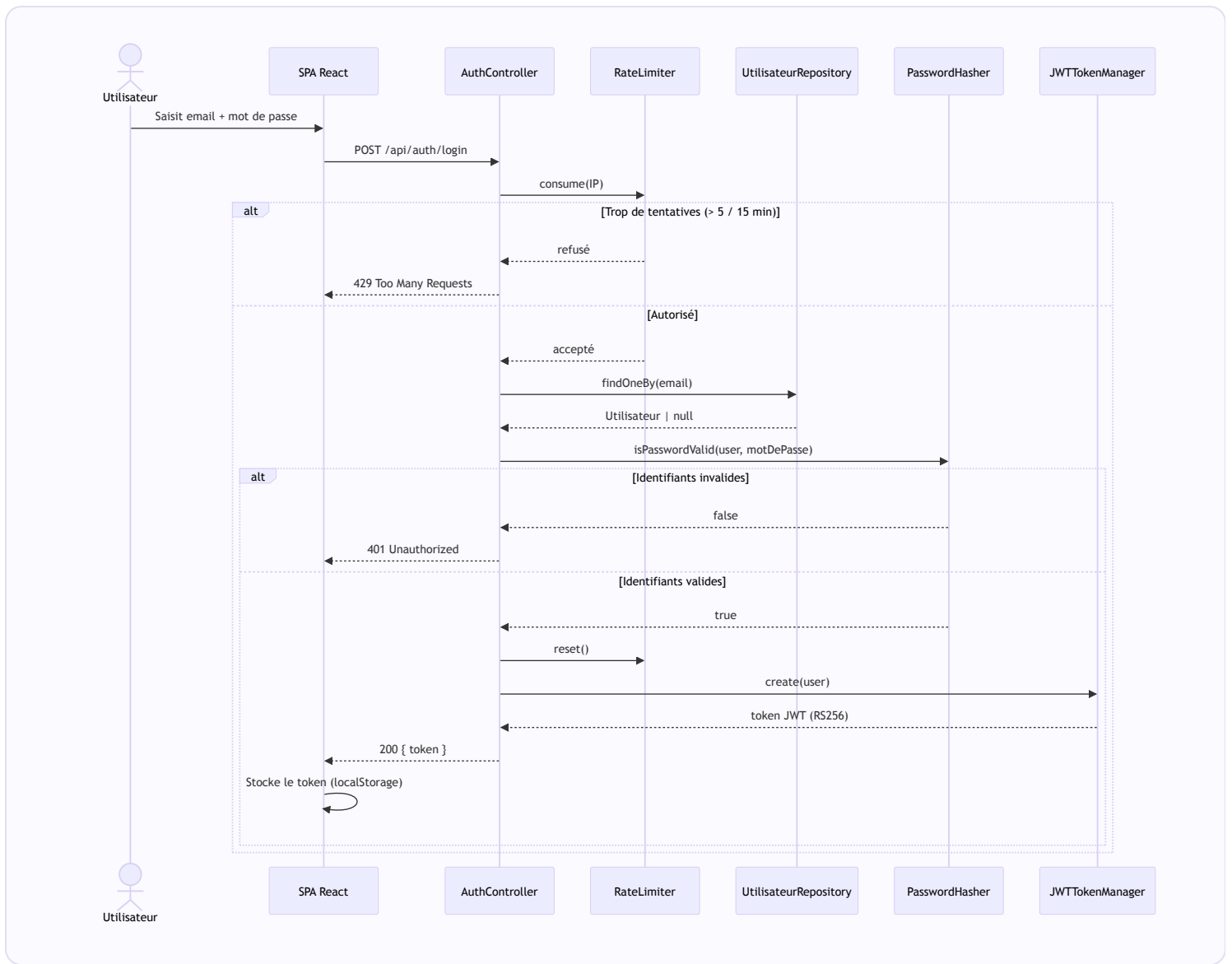
## 2.3 Description détaillée des cas d'utilisation principaux

| Cas d'utilisation       | Acteur         | Description                                                         | Endpoint API                              |
|-------------------------|----------------|---------------------------------------------------------------------|-------------------------------------------|
| S'authentifier          | Tous           | Connexion par email/mot de passe, obtention d'un jeton JWT          | POST /api/auth/login                      |
| Consulter mon véhicule  | Conducteur     | Affiche le véhicule affecté, son quota, sa position, son historique | GET /api/conducteur/mon-vehicule          |
| Signaler un problème    | Conducteur     | Crée une alerte destinée au gestionnaire                            | POST /api/conducteur/signaler-probleme    |
| Affecter un véhicule    | Gestionnaire   | Attribue un véhicule disponible à un conducteur                     | POST /api/gestionnaire/affecter           |
| Libérer un véhicule     | Gestionnaire   | Clôture une affectation, repasse le véhicule en disponible          | DELETE /api/gestionnaire/desaffected/{id} |
| Consulter la vue flotte | Gestionnaire   | Vue temps réel : qui conduit quoi, statuts, quotas                  | GET /api/gestionnaire/vue-flotte          |
| Gérer les véhicules     | Gestionnaire   | CRUD complet sur les véhicules                                      | /api/vehicules (API Platform)             |
| Gérer les utilisateurs  | Administrateur | CRUD sur les comptes                                                | /api/utilisateurs (API Platform)          |

## 3. Diagrammes de séquence

### 3.1 Authentification d'un utilisateur (login JWT)

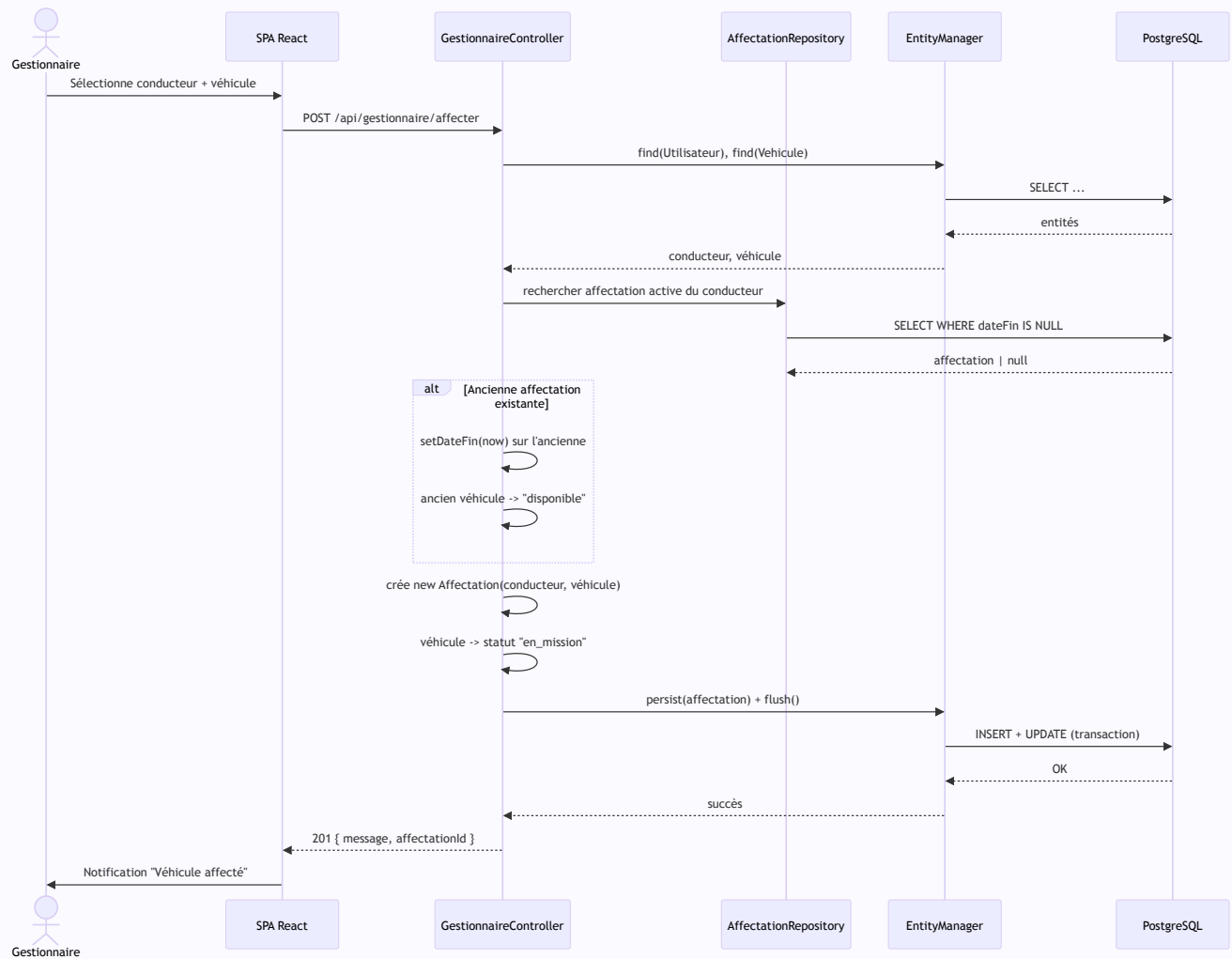
Ce scénario illustre la sécurisation de la connexion (protection brute force + vérification du mot de passe haché + génération du jeton).



### 3.2 Affectation d'un véhicule à un conducteur

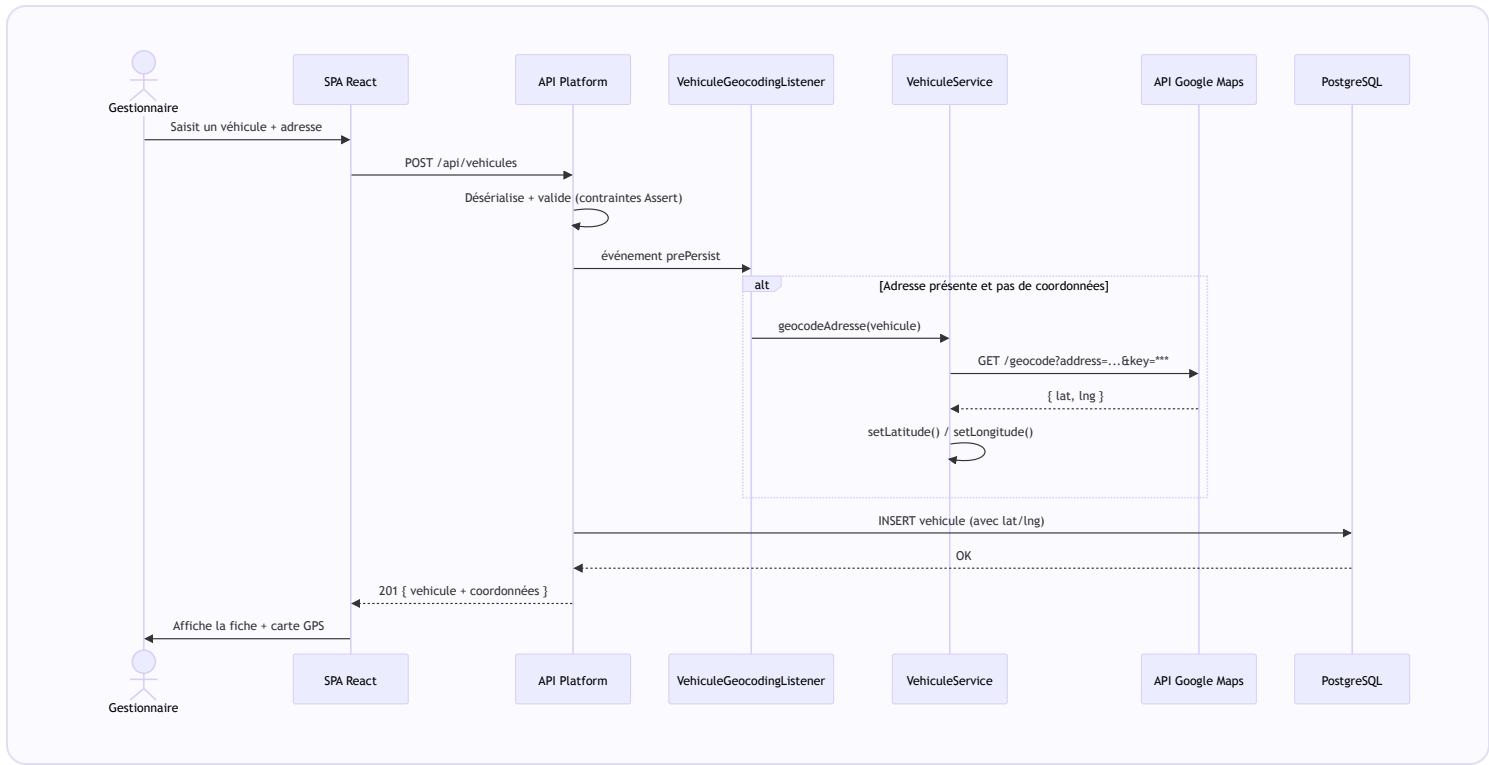
Scénario métier central : le gestionnaire affecte un véhicule. Le système clôture automatiquement l'ancienne affectation et synchronise les statuts.





### 3.3 Création d'un véhicule avec géolocalisation automatique (API externe)

Scénario illustrant l'intégration de l'API externe Google Maps via un **listener Doctrine** transparent.

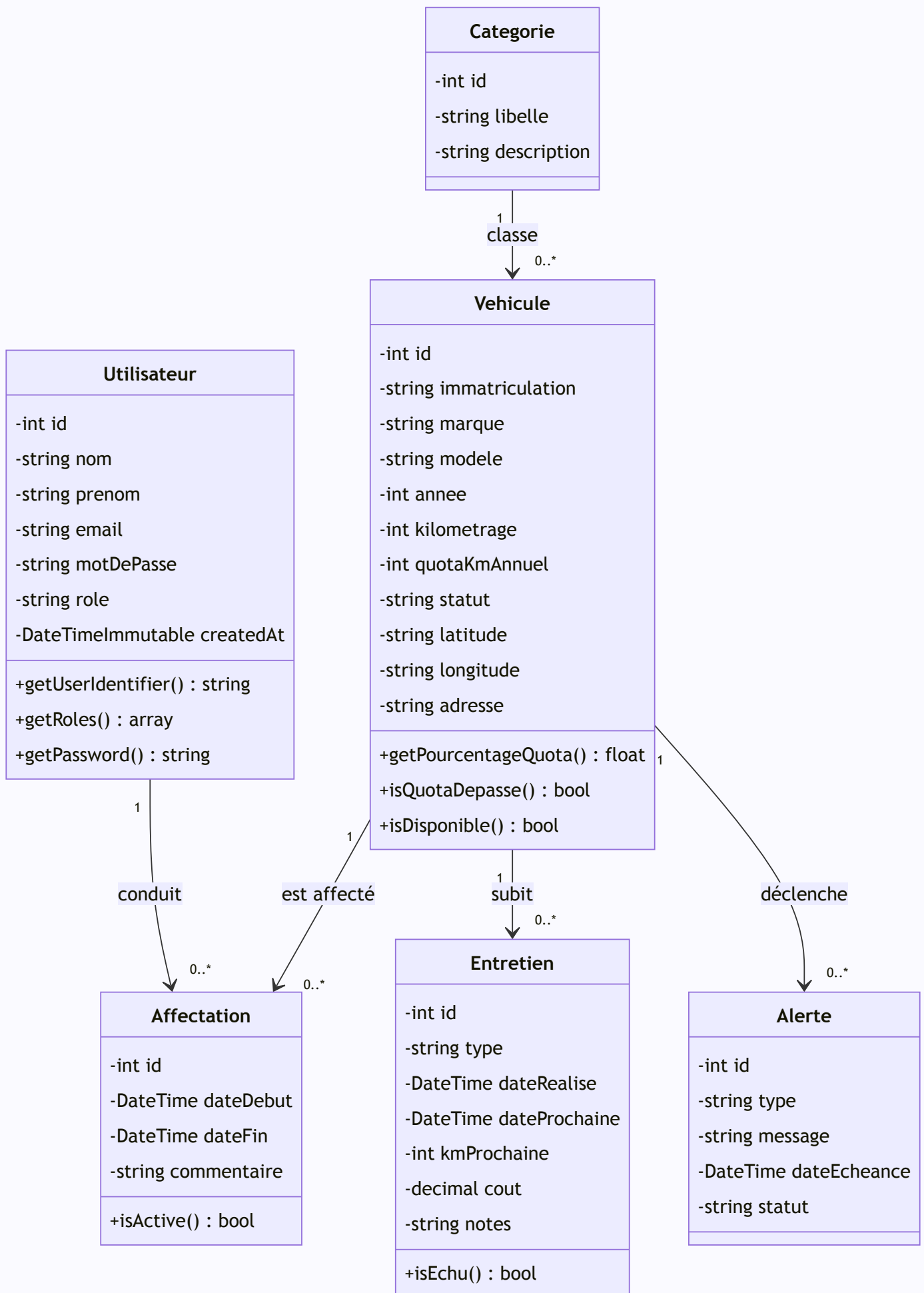


## 4. Diagramme de classes

### 4.1 Modèle du domaine (Entités / Couche Modèle)

Le diagramme ci-dessous représente les entités métier issues du MCD/MLD (Jalon 3), enrichies de leurs méthodes de logique applicative.



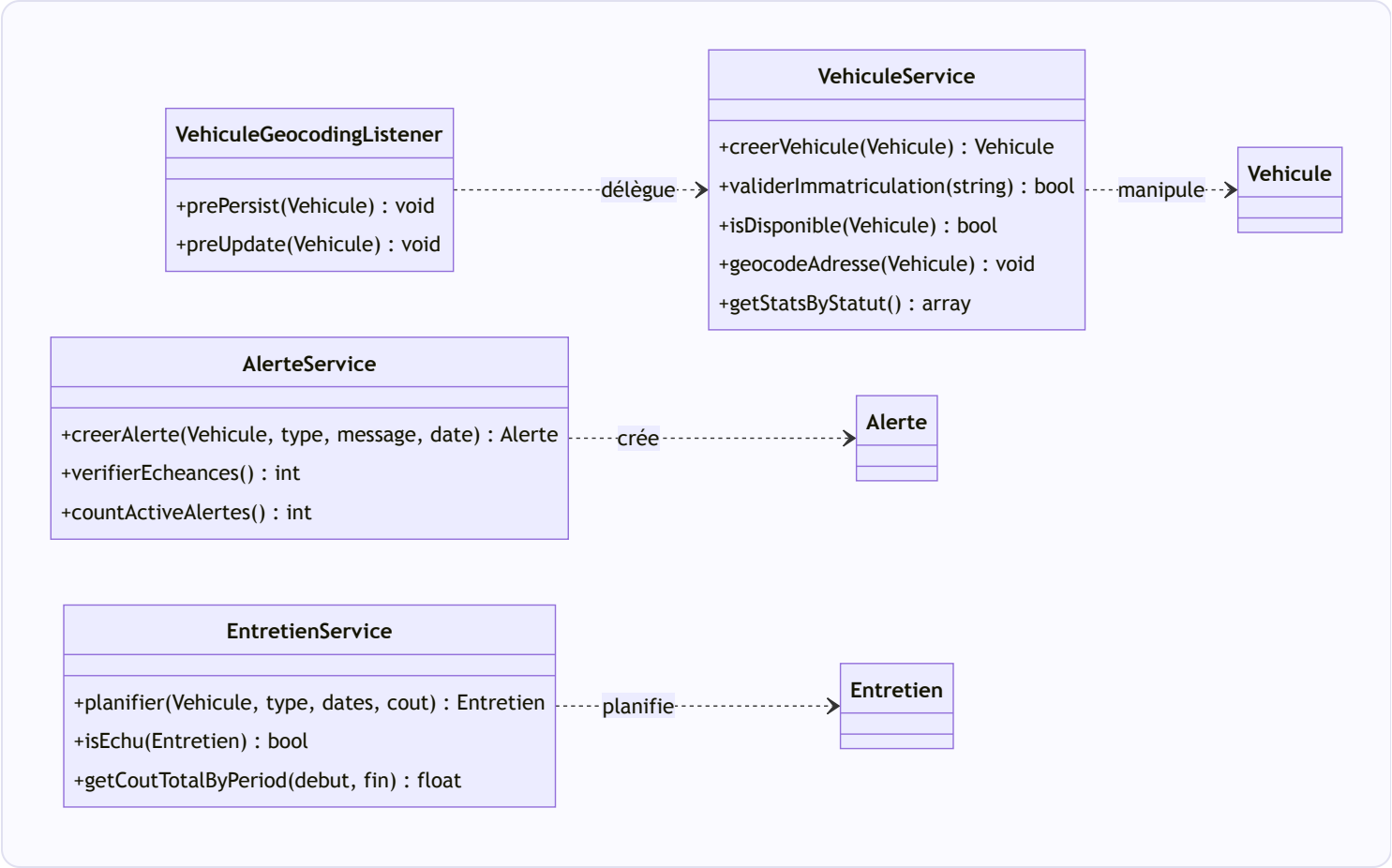


## 4.2 Lecture des relations et cardinalités

| Relation                  | Type              | Cardinalité                  | Signification                                                              |
|---------------------------|-------------------|------------------------------|----------------------------------------------------------------------------|
| Utilisateur → Affectation | Association (1-N) | 1 conducteur, N affectations | Un conducteur peut avoir plusieurs affectations dans le temps (historique) |
| Vehicule → Affectation    | Association (1-N) | 1 véhicule, N affectations   | Un véhicule passe par plusieurs conducteurs au fil du temps                |
| Categorie → Vehicule      | Association (1-N) | 1 catégorie, N véhicules     | Une catégorie (Berline, SUV...) regroupe plusieurs véhicules               |
| Vehicule → Entretien      | Composition (1-N) | 1 véhicule, N entretiens     | Les entretiens n'existent pas sans véhicule                                |
| Vehicule → Alerte         | Composition (1-N) | 1 véhicule, N alertes        | Les alertes sont rattachées à un véhicule                                  |

À propos d' **Affectation** : cette entité est une **classe d'association** entre **Utilisateur** (conducteur) et **Vehicule** . Elle porte des attributs propres ( **dateDebut** , **dateFin** , **commentaire** ) et une logique métier ( **isActive()** : une affectation est active si **dateFin** est nulle ou future). Elle matérialise le cœur du domaine : « qui conduit quel véhicule, et depuis quand ».

### 4.3 Couche applicative (Services) et organisation en packages



Chaque service respecte le principe de **responsabilité unique (SRP)** : **VehiculeService** ne gère que les véhicules, **AlerteService** que les alertes, etc.

## 5. Architecture multi-couches (Chapitre VIII)

### 5.1 Pattern MVC (adapté à une API REST)

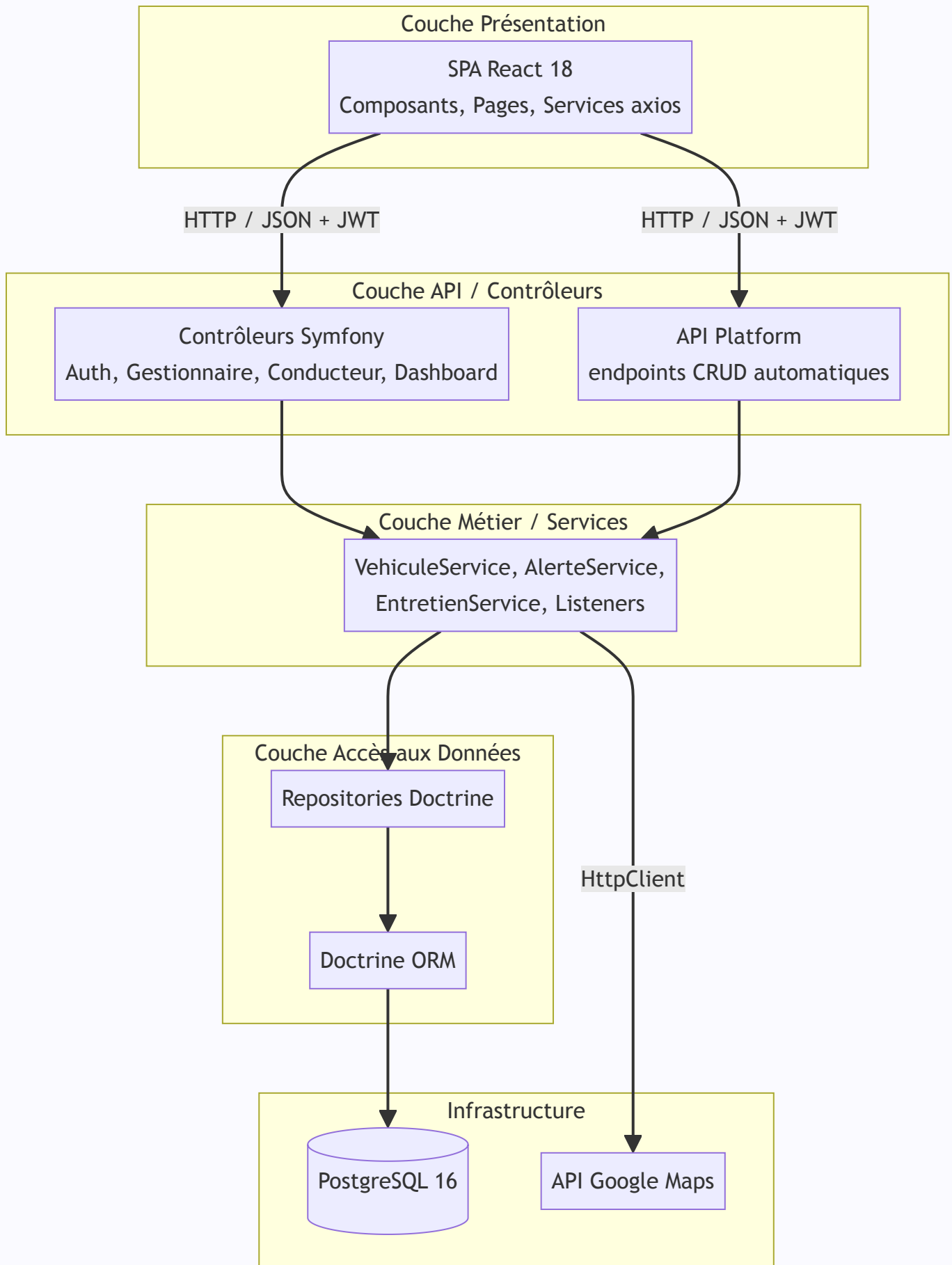
OptiFleet applique le pattern **MVC** dans sa déclinaison API REST + SPA :

| Couche MVC | Implémentation OptiFleet                                                    | Dossier                                   |
|------------|-----------------------------------------------------------------------------|-------------------------------------------|
| Modèle     | Entités Doctrine ( <b>Vehicule</b> , <b>Affectation</b> ...) + Repositories | <b>src/Entity</b> , <b>src/Repository</b> |
| Vue        | Réponses JSON (sérialisées par API Platform) + interface React              | API JSON / <b>frontend/src</b>            |
| Contrôleur | Contrôleurs Symfony qui reçoivent les requêtes HTTP                         | <b>src/Controller</b>                     |

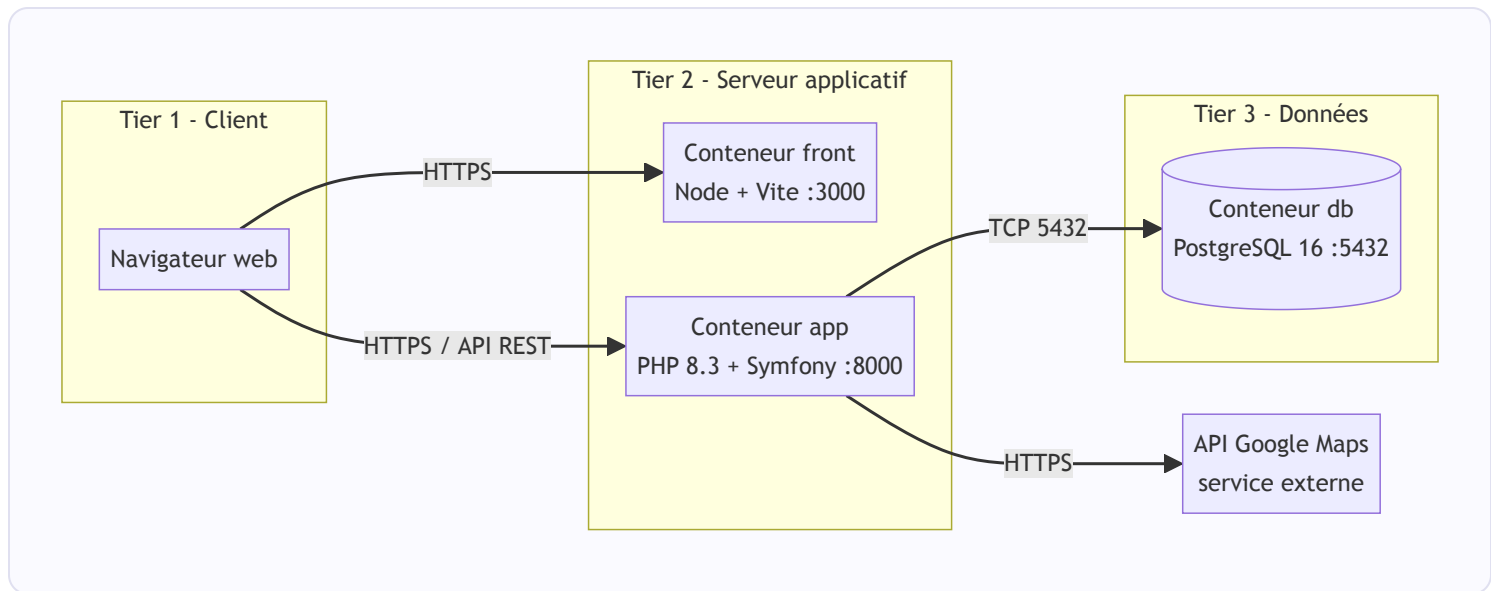
La logique métier complexe n'est **pas** placée dans les contrôleurs mais déléguée à la **couche Service** ( [src/Service](#) ), maintenant les contrôleurs « fins » (thin controllers). Le contrôleur orchestre ; le service décide.

## 5.2 Découpage logique en couches





## 5.3 Architecture n-tiers (déploiement physique)



**Distinction couche logique / tier physique** : l'application suit une architecture **3-tiers logique** (présentation / métier / données). Physiquement, ces couches sont conteneurisées via **Docker Compose** en trois services ( `front` , `app` , `db` ). En développement, les trois conteneurs tournent sur le même hôte ; en production, ils pourraient être répartis sur plusieurs machines sans changer l'architecture logique.

## 5.4 Séparation des responsabilités & bonnes pratiques

- **SRP (Single Responsibility Principle)** : chaque service a une responsabilité unique ( `VehiculeService` ne traite que les véhicules ).
- **Thin controllers** : les contrôleurs orchestrent, la logique réside dans les services.
- **Injection de dépendances** : tous les services et repositories sont injectés via l'autowiring Symfony (jamais de `new` sur une dépendance).
- **Configuration isolée** : tous les secrets (clé JWT, clé Google Maps, identifiants BDD) sont externalisés dans des variables d'environnement ( `.env` , non versionné ).
- **Routage centralisé** : déclaré par attributs PHP 8 ( `#[Route(...)]` ) directement sur les méthodes des contrôleurs.
- **Conventions PSR** : code conforme PSR-12, vérifié automatiquement par php-cs-fixer en CI.

## 5.5 Design patterns mis en œuvre

| Pattern                   | Où                                      | Rôle                                                                                                                             |
|---------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Repository                | <code>src/Repository</code>             | Encapsule l'accès aux données (requêtes Doctrine)                                                                                |
| Service Layer             | <code>src/Service</code>                | Centralise la logique métier réutilisable                                                                                        |
| Observer / Event Listener | <code>VehiculeGeocodingListener</code>  | Réagit aux événements Doctrine ( <code>prePersist</code> / <code>preUpdate</code> ) pour géolocaliser sans coupler le contrôleur |
| Dependency Injection      | Conteneur Symfony                       | Découple les composants et facilite les tests                                                                                    |
| DTO / Sérialisation       | API Platform (groupes de normalisation) | Contrôle finement les données exposées par l'API                                                                                 |

## 6. Composants externes et bibliothèques

### 6.1 Back-end (bundles Symfony)

| Bibliothèque                                             | Rôle dans l'architecture                                                   |
|----------------------------------------------------------|----------------------------------------------------------------------------|
| <code>api-platform/core</code>                           | Génère automatiquement les endpoints REST CRUD à partir des entités        |
| <code>doctrine/orm</code> + <code>doctrine-bundle</code> | ORM : mapping objet-relationnel, couche d'accès aux données                |
| <code>doctrine-migrations-bundle</code>                  | Versionnement du schéma de base de données                                 |
| <code>lexik/jwt-authentication-bundle</code>             | Authentification par jeton JWT (signature RS256)                           |
| <code>nelmio/cors-bundle</code>                          | Gestion des en-têtes CORS (autorisation des appels cross-origin de la SPA) |
| <code>symfony/rate-limiter</code>                        | Limitation des tentatives de connexion (anti brute force)                  |
| <code>symfony/http-client</code>                         | Appels HTTP sortants vers l'API Google Maps                                |
| <code>symfony/mailer</code>                              | Envoi d'e-mails (notifications, prévu)                                     |

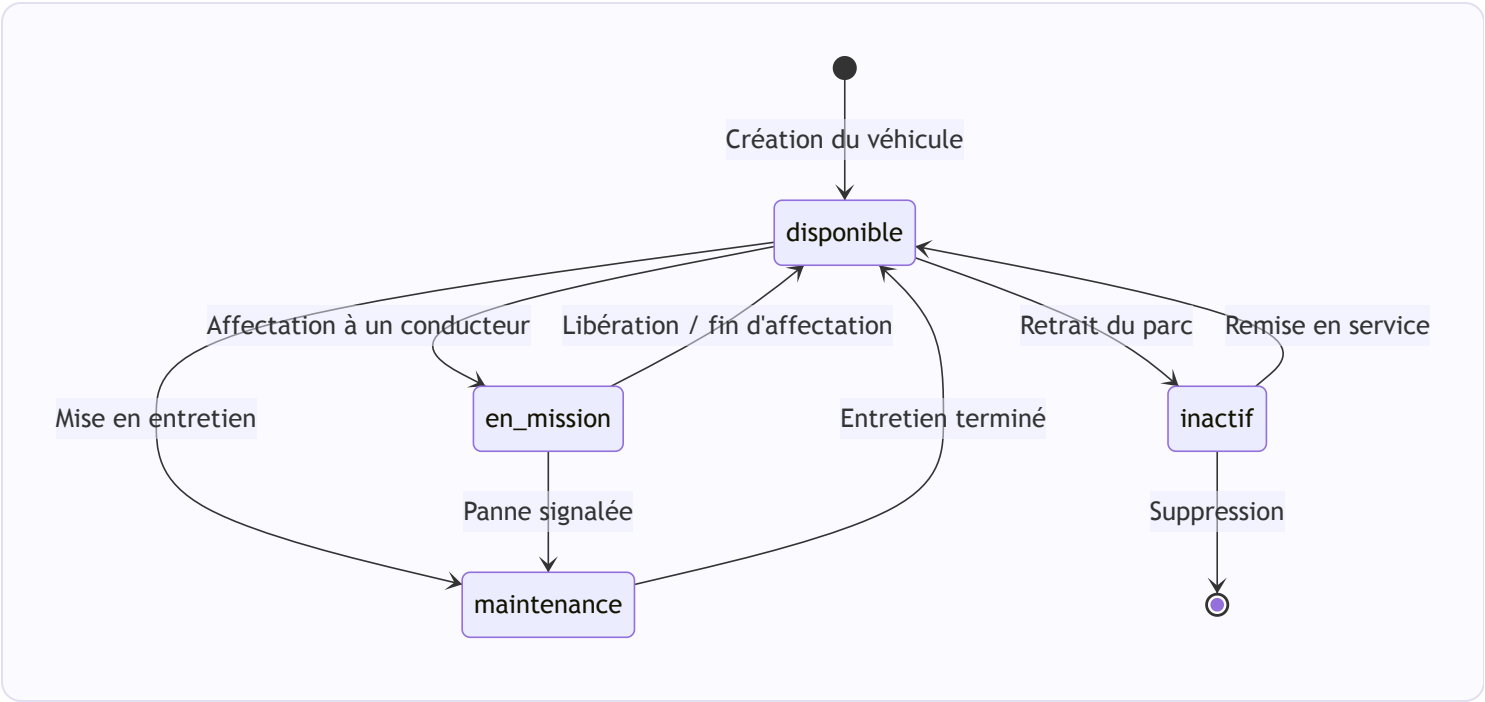
6.2 Front-end (packages npm)

| Bibliothèque      | Rôle                                                             |
|-------------------|------------------------------------------------------------------|
| react + react-dom | Bibliothèque d'interface (SPA)                                   |
| react-router-dom  | Routage côté client + protection des routes par rôle             |
| axios             | Client HTTP (appels à l'API, injection automatique du jeton JWT) |
| zustand           | Gestion d'état global léger (authentification, notifications)    |
| recharts          | Graphiques du tableau de bord (KPIs)                             |
| tailwindcss       | Framework CSS utilitaire (design system)                         |

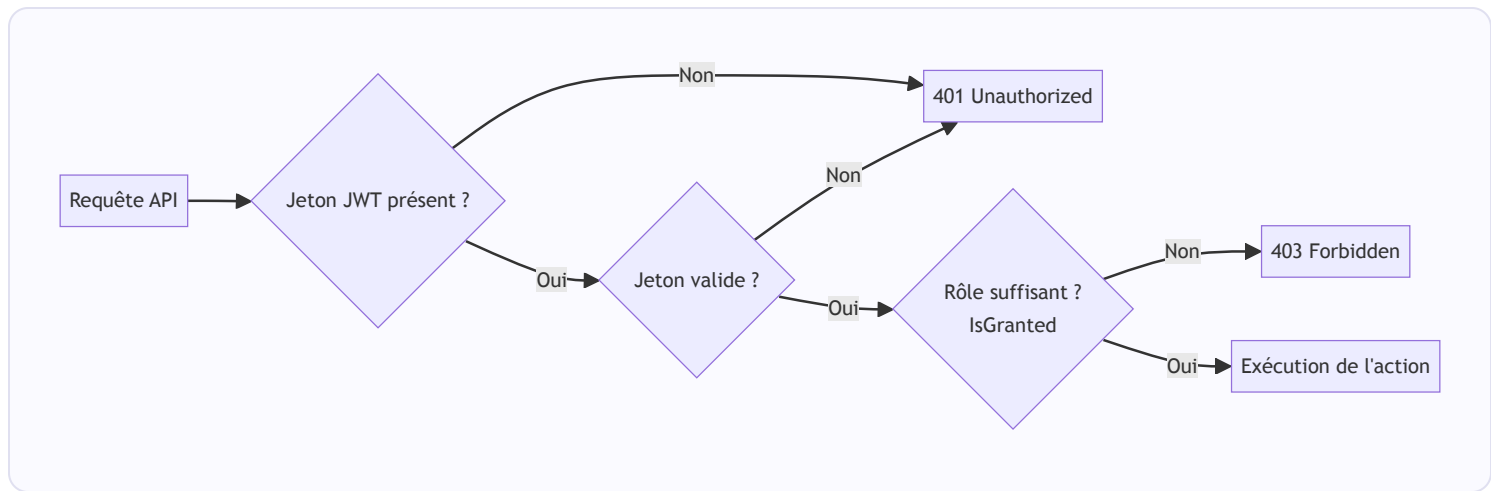
7. Schémas complémentaires

7.1 Cycle de vie du statut d'un véhicule (machine à états)

Le statut d'un véhicule évolue selon les affectations et la maintenance. Ce diagramme d'états clarifie un workflow central de l'application.



## 7.2 Flux d'authentification et d'autorisation



## Conclusion

Cette conception technique fournit une vision claire et cohérente de l'application :

- les **cas d'utilisation** couvrent l'ensemble des fonctionnalités du CDCF, organisés par rôle ;
- les **diagrammes de séquence** détaillent les scénarios critiques (authentification sécurisée, affectation, géolocalisation) ;
- le **diagramme de classes** reflète fidèlement le modèle du domaine implémenté ;
- l'**architecture multi-couches** (MVC logique + 3-tiers physique conteneurisé) garantit la séparation des responsabilités et la maintenabilité.

Ces plans ont directement guidé le développement : les entités, services et contrôleurs présentés ici correspondent à l'implémentation réelle du dépôt Git, validant la cohérence entre conception et code.